

Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Catedrático: Erick Marroquín



Festival Universitario de Música

Análisis y Diseño

Jared Eduardo Guerra Aguirre

Carné: 26502

Ciencia de la Computación y Tecnologías de la Información

Sección 11

07 de septiembre de 2026

Requisitos funcionales

1. El sistema debe permitir crear un nuevo festival solicitando como mínimo su nombre, código de identificación y nombre del coordinador. Al crear un nuevo festival, se deben eliminar los datos del festival anterior e iniciar sin escenarios ni artistas registrados.
2. El sistema debe administrar un máximo de cinco escenarios mediante un arreglo básico de objetos Escenario, permitiendo que las posiciones que todavía no tengan un escenario configurado contengan null.
3. El sistema debe permitir configurar un nuevo escenario indicando la posición del arreglo y proporcionando su código, nombre, ubicación, capacidad máxima de asistentes y estado.
4. El sistema debe validar que la posición seleccionada para configurar un escenario se encuentre dentro de los límites del arreglo y esté disponible antes de almacenar el objeto.
5. El sistema debe validar que la capacidad máxima de un escenario sea mayor que cero. Si se intenta crear o modificar un escenario con una capacidad menor o igual que cero, debe producirse una `IllegalArgumentException`.
6. El sistema debe permitir consultar todos los escenarios configurados, mostrando únicamente las posiciones que contienen objetos Escenario y evitando mostrar las posiciones que contienen null.
7. El sistema debe permitir consultar un escenario específico mediante su posición en el arreglo. Si la posición no existe o contiene null, debe informarse al usuario sin finalizar inesperadamente el programa.
8. El sistema debe permitir modificar la capacidad máxima y el estado de un escenario previamente configurado, validando que la posición no contenga null y que la nueva capacidad sea mayor que cero.
9. El sistema debe permitir retirar un escenario previamente configurado, asignando nuevamente null a la posición correspondiente para dejarla disponible.
10. El sistema debe administrar una cantidad dinámica de artistas mediante un `ArrayList<Artista>`.
11. El sistema debe permitir registrar un artista proporcionando su código, nombre artístico, género musical, duración de la presentación y cantidad estimada de asistentes.
12. El sistema debe validar que la duración de la presentación de un artista sea mayor que cero y que su cantidad estimada de asistentes no sea negativa. Los valores que incumplan estas condiciones deben producir una `IllegalArgumentException`.

13. El sistema debe impedir que se registren dos artistas con el mismo código.
14. El sistema debe permitir consultar todos los artistas registrados y manejar correctamente el caso en que el ArrayList se encuentre vacío.
15. El sistema debe permitir buscar un artista mediante su código, recorriendo para ello el ArrayList<Artista>.
16. El sistema debe permitir modificar la información de un artista previamente registrado, aplicando a los nuevos valores las mismas validaciones utilizadas durante su creación.
17. El sistema debe permitir cancelar la participación de un artista, buscándolo mediante su código y eliminándolo del ArrayList.
18. El sistema debe calcular la cantidad de escenarios configurados y la cantidad de espacios disponibles dentro del arreglo.
19. El sistema debe determinar cuál de los escenarios configurados posee la mayor capacidad máxima de asistentes.
20. El sistema debe calcular la cantidad total de artistas registrados.
21. El sistema debe determinar cuál artista posee la presentación de mayor duración.
22. El sistema debe determinar cuál artista posee la mayor cantidad estimada de asistentes.
23. El sistema debe calcular el promedio de duración de las presentaciones registradas y evitar realizar cálculos que requieran artistas cuando el ArrayList esté vacío.
24. El sistema debe manejar mediante InputMismatchException los casos en que el usuario introduzca un dato incorrecto cuando Scanner solicite un valor numérico, limpiando la entrada incorrecta y permitiendo continuar utilizando el programa.
25. El sistema debe utilizar bloques try-catch para manejar las excepciones producidas durante la interacción con el usuario y las validaciones de los objetos.
26. El sistema debe utilizar al menos un bloque finally para ejecutar una acción final independientemente de que una operación termine correctamente o produzca una excepción.
27. El sistema debe continuar funcionando después de una entrada incorrecta o de cualquier excepción que haya sido manejada correctamente.
28. El sistema debe presentar un menú que permita crear un nuevo festival, configurar, consultar, modificar y retirar escenarios; registrar, consultar, buscar, modificar y cancelar artistas; mostrar el reporte del festival y salir del programa.

1. ¿Qué propiedades y métodos tendrá cada clase?

La clase Escenario tendrá las propiedades codigo, nombre, ubicacion, capacidadMaxima y estado. La clase Artista tendrá codigo, nombreArtistico, generoMusical, duracionPresentacion y cantidadEstimadaAsistentes. La clase Festival tendrá nombre, codigoidentificacion, nombreCoordinador, un arreglo escenarios y un ArrayList llamado artistas. Los métodos de Escenario y Artista permitirán consultar y modificar sus datos, Festival tendrá los métodos para configurar, consultar, modificar y quitar escenarios, registrar, buscar, modificar y eliminar artistas, y calcular los datos requeridos para el reporte. Main tendrá el método main() y métodos para mostrar el menú y controlar la interacción con el usuario.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Los datos de texto, como códigos, nombres, ubicación, género y estado, se guardarán como String, algunos códigos podrían contener únicamente números, pero utilizar String permite tratarlos como identificadores y no como cantidades en las que se realizan operaciones matemáticas. La capacidad máxima, duración y cantidad estimada de asistentes utilizarán int, porque son cantidades enteras. El arreglo de escenarios será de tipo Escenario[] y la colección dinámica será ArrayList<Artista>. Los métodos usarán tipos de retorno según con su función, como Escenario o Artista cuando realizan una búsqueda, int y double, para cálculos, boolean para decir si una operación tuvo éxito y void cuando solo cambian el estado del objeto.

3. ¿Cuál propiedad debe implementarse utilizando un arreglo básico? ¿Qué objetos almacenará y cuál será su tamaño?

La propiedad escenarios de la clase Festival debe implementarse mediante un arreglo básico de tipo Escenario[] con un tamaño fijo de 5 posiciones, porque cada festival puede contar con un máximo de cinco escenarios. Cada posición almacenará una referencia a un objeto Escenario o tendrá el valor null cuando ese espacio todavía no haya sido ocupado. Esta estructura permite acceder a cada escenario mediante su índice y controlar de forma directa cuáles espacios se encuentran ocupados o disponibles.

4. ¿Cuál propiedad debe implementarse utilizando un ArrayList? ¿Qué objetos almacenará?

La propiedad artistas de Festival debe implementarse como ArrayList<Artista>, porque la cantidad de artistas puede cambiar durante la organización del festival. El ArrayList almacenará objetos de tipo Artista y permitirá agregar nuevos participantes y eliminarlos si cancelan su participación.

5. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

Los atributos de Escenario, Artista y Festival serán private por el encapsulamiento y evitar que otras clases modifiquen directamente el estado interno de los objetos. Los constructores, getters, setters y operaciones que deban ser utilizadas desde otras clases serán public. Los métodos que solamente sean necesarios para el funcionamiento interno de una clase serán private. En Main, los métodos también serán private static, main() será public static.

6. ¿Qué parámetros serán requeridos por los métodos en sus clases?

Los constructores recibirán los datos para inicializar cada objeto. Escenario recibirá código, nombre, ubicación, capacidad máxima y estado. Artista recibirá código, nombre artístico, género musical, duración y cantidad estimada de asistentes. Festival recibirá nombre, código de identificación y nombre del coordinador. Los métodos de escenarios recibirán una posición del arreglo y si les corresponde, un objeto Escenario, una capacidad o un estado. Los métodos de artistas recibirán un objeto Artista para registrarlo o un código para buscarlo, modificarlo o eliminarlo. Los setters recibirán como parámetro el nuevo valor que se desea asignar.

7. ¿Cómo proveerá valores iniciales a sus objetos? ¿Qué valores deberán validarse?

Los valores iniciales serán dados mediante constructores. Al crear un Festival, su arreglo Escenario[] se inicializará con cinco posiciones que inicialmente contendrán null y su ArrayList<Artista> comenzará vacío. Al crear o modificar un Escenario, se verificará que capacidadMaxima sea mayor que cero, si no, se lanzará IllegalArgumentException. En Artista, duracionPresentacion deberá ser mayor que cero y cantidadEstimadaAsistentes deberá ser mayor o igual que cero, lanzándose también IllegalArgumentException cuando estas reglas no se cumplan.

8. ¿Cómo determinará si una posición del arreglo contiene un escenario o contiene null?

Antes de acceder a los métodos o propiedades de un escenario se verificará que la posición solicitada se encuentre entre 0 y escenarios.length - 1. Después se evaluará escenarios[posicion] != null para ver si existe un escenario configurado. Si el resultado es false significa que la posición contiene null y está disponible. Esto deberá realizarse antes de consultar, modificar o retirar un escenario para no usar una referencia null.

9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

Las operaciones sobre artistas se realizarán recorriendo el ArrayList<Artista> con estructuras repetitivas. Para buscar un artista se comparará el código solicitado con el código de cada objeto hasta encontrarlo. Para modificarlo, primero se localizará el objeto y después se utilizarán sus setters para cambiar la información, manteniendo las mismas validaciones utilizadas en su creación. Para cancelar una participación, se localizará al artista por su código y se eliminará del ArrayList. Antes de registrar un artista nuevo también se realizará una búsqueda para impedir que existan dos artistas con el mismo código.

10. ¿Qué situaciones pueden producir excepciones? ¿Dónde se utilizarán try-catch y finally?

Las excepciones serán InputMismatchException e IllegalArgumentException. InputMismatchException puede ocurrir en Main cuando Scanner espera un valor numérico y el usuario introduce otro tipo de dato, el catch deberá limpiar la entrada incorrecta para permitir que el programa continúe. IllegalArgumentException se producirá cuando se intente crear o modificar un escenario con capacidad menor o igual que cero, o un artista con duración menor o igual que cero o asistentes negativos. Los try-catch se utilizarán en las operaciones de entrada y creación o modificación de objetos. También se incluirá un bloque finally, por ejemplo al finalizar un intento de operación del menú, para ejecutar una acción independientemente de que la operación haya tenido éxito o producido una excepción.

Clase Escenario

Tabla de atributos

Clase	Atributos	Tipo de dato	Visibilidad	Propósito
Escenario	codigo	String	private	Identificar el escenario mediante un código.
Escenario	nombre	String	private	Guardar el nombre del escenario.
Escenario	ubicacion	String	private	Guardar la ubicación física del escenario.
Escenario	capacidadMaxima	int	private	Almacenar la cantidad máxima de asistentes permitida.
Escenario	estado	String	private	Indicar el estado actual del escenario.

Tabla de métodos

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Escenario	Escenario	-	public	String codigo, String nombre, String ubicacion, int capacidadMaxima, String estado	Crear un escenario con sus valores iniciales y validar que la capacidad sea mayor que cero. Lanza IllegalArgumentException si la capacidad es menor o igual que cero.
Escenario	getCodigo	String	public	-	Obtener el código.
Escenario	getNombre	String	public	-	Obtener el nombre.
Escenario	getUbicacion	String	public	-	Obtener la ubicación.
Escenario	getCapacidadMaxima	int	public	-	Obtener la capacidad máxima.
Escenario	getEstado	String	public	-	Obtener el estado.
Escenario	setCapacidadMaxima	void	public	int capacidadMaxima	Cambiar la capacidad. Lanza IllegalArgumentException si es menor o igual que cero.
Escenario	setEstado	void	public	String estado	Modificar el estado del escenario.
Escenario	toString	String	public	-	Devolver la información completa del escenario.

Clase Artista

Tabla de atributos

Clase	Atributos	Tipo de dato	Visibilidad	Propósito
Artista	codigo	String	private	Identificar de manera única al artista.
Artista	nombreArtistico	String	private	Guardar el nombre artístico.
Artista	generoMusical	String	private	Guardar el género musical.
Artista	duracionPresentacion	int	private	Guardar la duración de la presentación en minutos.
Artista	cantidadEstimadaAsistentes	int	private	Guardar la cantidad estimada de asistentes.

Tabla de métodos

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Artista	Artista	-	public	String codigo, String nombreArtistico, String generoMusical, int duracionPresentacion, int cantidadEstimadaAsistentes	Crear un artista con sus valores iniciales y validar la duración y cantidad estimada de asistentes. Lanza IllegalArgumentException si la duración es menor o igual que cero o la cantidad estimada de asistentes es negativa.
Artista	getCodigo	String	public	-	Obtener el código del artista.
Artista	getNombreArtistico	String	public	-	Obtener el nombre artístico.
Artista	getGeneroMusical	String	public	-	Obtener el género musical.
Artista	getDuracionPresentacion	int	public	-	Obtener la duración de la presentación.
Artista	getCantidadEstimadaAsistentes	int	public	-	Obtener la cantidad estimada de asistentes.
Artista	setNombreArtistico	void	public	String nombreArtistico	Modificar el nombre artístico.
Artista	setGeneroMusical	void	public	String generoMusical	Modificar el género musical.
Artista	setDuracionPresentacion	void	public	int duracionPresentacion	Modificar la duración. Lanza IllegalArgumentException si es menor o igual que cero.
Artista	setCantidadEstimadaAsistentes	void	public	int cantidad	Modificar los asistentes estimados. Lanza IllegalArgumentException si es negativo.
Artista	toString	String	public	-	Devolver la información completa del artista.

Clase Festival

Tabla de atributos

Clase	Atributos	Tipo de dato	Visibilidad	Propósito
Festival	nombre	String	private	Guardar el nombre del festival.
Festival	codigoidentificacion	String	private	Guardar el código de identificación del festival.
Festival	nombreCoordinador	String	private	Guardar el nombre del coordinador.
Festival	escenarios	Escenario[]	private	Almacenar hasta 5 escenarios utilizando un arreglo básico.
Festival	artistas	ArrayList<Artista>	private	Almacenar dinámicamente los artistas registrados.

Tabla de métodos

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Festival	Festival	-	public	String nombre, String codigoidentificacion, String nombreCoordinador	Crear el festival e inicializar Escenario[5] y ArrayList< Artista>.
Festival	getNombre	String	public	-	Obtener el nombre del festival.
Festival	getCodigoidentificacion	String	public	-	Obtener el código del festival.
Festival	getNombreCoordinador	String	public	-	Obtener el coordinador.
Festival	configurarEscenario	boolean	public	int posicion, Escenario escenario	Guardar un escenario si la posición es válida y está disponible.
Festival	obtenerEscenario	Escenario	public	int posicion	Obtener el escenario almacenado en una posición.
Festival	modificarEscenario	boolean	public	int posicion, int capacidadMaxima, String estado	Modificar capacidad y estado de un escenario existente.
Festival	retirarEscenario	boolean	public	int posicion	Quitar el escenario y volver a poner null en la posición.
Festival	registrarArtista	boolean	public	Artista artista	Agregar un artista si su código no está repetido.
Festival	mostrarArtistas	void	public	-	Mostrar todos los artistas registrados recorriendo el ArrayList. Si no existen artistas, informar que no hay artistas registrados.
Festival	buscarArtista	Artista	public	String codigo	Recorrer el ArrayList y devolver el artista correspondiente.
Festival	eliminarArtista	boolean	public	String codigo	Localizar y eliminar un artista del ArrayList.
Festival	cantidadEscenariosConfigurados	int	public	-	Calcular cuántas posiciones contienen escenarios.
Festival	cantidadEspaciosDisponibles	int	public	-	Calcular cuántas posiciones contienen null.
Festival	escenarioMayorCapacidad	Escenario	public	-	Encontrar el escenario con mayor capacidad.
Festival	cantidadArtistas	int	public	-	Obtener la cantidad de artistas registrados.
Festival	artistaMayorDuracion	Artista	public	-	Encontrar el artista con la presentación más larga.
Festival	artistaMayorAsistencia	Artista	public	-	Encontrar el artista con mayor cantidad estimada de asistentes.
Festival	promedioDuracion	double	public	-	Calcular el promedio de duración de las presentaciones.

Clase Main

Tabla de atributos

Clase	Atributos	Tipo de dato	Visibilidad	Propósito
Main	scanner	Scanner	private static	Leer las entradas ingresadas por el usuario.
Main	festival	Festival	private static	Mantener la instancia actual del festival administrado por el programa.

Tabla de métodos

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Main	main	void	public static	String[] args	Iniciar el programa y controlar el ciclo principal del menú.
Main	mostrarMenu	void	private static	-	Mostrar las 13 opciones disponibles.
Main	crearFestival	void	private static	-	Solicitar los datos y reemplazar la instancia actual del festival.
Main	configurarEscenario	void	private static	-	Solicitar posición y datos para crear un escenario.
Main	consultarEscenarios	void	private static	-	Mostrar los escenarios configurados.
Main	consultarEscenario	void	private static	-	Solicitar una posición y mostrar su escenario.
Main	modificarEscenario	void	private static	-	Solicitar los nuevos datos de un escenario.
Main	retirarEscenario	void	private static	-	Solicitar una posición y retirar su escenario.
Main	registrarArtista	void	private static	-	Solicitar datos y registrar un artista.
Main	consultarArtistas	void	private static	-	Mostrar todos los artistas registrados.
Main	buscarArtista	void	private static	-	Solicitar un código y mostrar el artista encontrado.
Main	modificarArtista	void	private static	-	Buscar un artista y solicitar sus nuevos datos.
Main	cancelarParticipacion	void	private static	-	Solicitar el código y eliminar al artista.
Main	mostrarReporte	void	private static	-	Mostrar todos los cálculos requeridos del festival.

Clasificación MVC

Clase	Rol	Propósito
Escenario	Modelo	Representa un escenario del festival con su código, nombre, ubicación, capacidad máxima y estado, y valida los datos correspondientes.
Artista	Modelo	Representa a un artista participante con su código, nombre artístico, género musical, duración de presentación y cantidad estimada de asistentes, y valida sus datos.
Festival	Modelo	Representa y administra el festival, almacena los escenarios con un arreglo básico y los artistas con un ArrayList, y realiza las operaciones y cálculos.
Main	Driver / Controller / View	Contiene el método main(), administra el festival activo, muestra el menú y los resultados, recibe las entradas del usuario, maneja las excepciones y ejecuta las operaciones mediante la clase Festival.

Diagrama de clases

